

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Алгоритмы и структуры данных»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
«Сортировка распределением (карманная сортировка)»

Выполнил:
Пахомов Владимир Юрьевич, студент группы N3250

(подпись)

Проверил:
Ерофеев Сергей Анатольевич

(отметка о выполнении)

(подпись)

Санкт-Петербург
2026.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ВВЕДЕНИЕ.....	3
1. АЛГОРИТМ СОРТИРОВКИ.....	4
2. ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ПЕРЕМЕННЫХ.....	5
3. БЛОК-СХЕМА АЛГОРИТМА.....	6
4. ТЕСТИРОВАНИЕ АЛГОРИТМА.....	6
5. ИСХОДНЫЙ КОД АЛГОРИТМА.....	8
ЗАКЛЮЧЕНИЕ.....	12
ПРИЛОЖЕНИЕ А.....	13

ВВЕДЕНИЕ

Цель работы — разработать программу сортировки распределением (карманной сортировки) последовательности вещественных чисел, которая будет храниться в статическом или динамическом массиве по выбору пользователя, определить сложность алгоритма путём подсчёта количества элементарных операций, выполняемых в программе.

1. АЛГОРИТМ СОРТИРОВКИ

Программа поддерживает несколько опций при запуске:

- -s, --static=SIZE - использовать в программе статический массив размера SIZE;
- -d, --dynamic - использовать в программе динамический массив;
- -h, --help - вывод справки;
- -v, --version - вывод информации об авторе.

Наличие хотя бы одной из опций (-s или -d) является обязательным.

Далее происходит сортировка массива по следующему алгоритму:

- 1) Создаются карманы, количество карманов равно количеству элементов в массиве;
- 2) Далее элементы распределяются по карманам. Для начала определяются максимальный (max) и минимальный (min) элементы массива. Индекс нужного кармана для каждого элемента определяется по формуле:
$$\frac{(x - \min) \cdot (\text{len}(\text{array}) - 1)}{\max - \min}.$$
- 3) Каждый заполненный карман сортируется с помощью сортировки вставками;
- 4) Содержимое карманов последовательно записывается в исходный массив.

На выходе получаем отсортированный массив и направляем его в стандартный поток вывода.

2. ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ПЕРЕМЕННЫХ

Ниже представлена таблица с описанием использованных переменных.

Таблица 1 — Используемые переменные

Название	Тип	Границы типа	Назначение
longopts[]	struct option	-	Переменная, содержащая описание опций
size	int	от 0 до 2147483647	Размер исходного массива
isDynamic	int	-1, 0, 1	Флаг, определяющий тип массива
opt	int	от 0 до 2147483647	Используется для чтения опций из функции getopt_long
argc	int	от 1 до 2147483647	Хранит количество аргументов при запуске программы
argv[]	char**	адрес памяти	Массив переданных аргументов
endptr	char*	адрес памяти	Используется для работы функции strtol
dynamicArray	float*	адрес памяти	Указатель на выделенную память для динамического массива
n	int	от 0 до 2147483647	Размер массива
staticArray[]	float[size]	-	Статический массив размера size
filename	char*	адрес памяти	Ссылается на имя файла

3. БЛОК-СХЕМА АЛГОРИТМА

Блок-схема алгоритма представлена в Приложении А.

4. ТЕСТИРОВАНИЕ АЛГОРИТМА

```
> ./lab1 -d data.txt
Используется динамический массив

Исходный массив из файла:
3.000 43.000 4325.000 1.000 46.000 12.440 -123.340 21947840.000

Отсортированный массив:
-123.340 1.000 3.000 12.440 43.000 46.000 4325.000 21947840.000

> ./lab1 -s=4 data.txt
Используется статический массив размера 4

Исходный массив из файла:
3.000 43.000 4325.000 1.000

Отсортированный массив:
1.000 3.000 43.000 4325.000
```

Рисунок 1 — Результат успешной работы алгоритма

```
> ./lab1 -v
Карманная сортировка (Bucket Sort) v1.0
Информация об авторе:
Пахомов Владимир Юрьевич, гр. N3250, ИСУ 467036

> ./lab1 -h
Использование: ./lab1 [ОПЦИИ] <имя_файла>

Программа для сортировки чисел из файла методом карманной сортировки (Bucket Sort)

Опции:
  -s, --static=SIZE      Использовать статический массив (VLA) заданного
                           размера SIZE
  -d, --dynamic          Использовать динамический массив (автоопределе
                           ние размера файла)
  -h, --help             Показать эту справку и выйти
  -v, --version          Показать информацию о версии и выйти

Примеры:
./lab1 -s=100 input.txt
./lab1 -d input.txt
```

Рисунок 2: Результат работы алгоритма с опциями -v и -h

```
> ./lab1 -d text.txt
Ошибка: не удалось открыть файл text.txt
```

```
> ./lab1 -d
Ошибка: не указано имя файла
```

```
> ./lab1 -f
./lab1: invalid option -- 'f'
Использование: ./lab1 [-s=size | -d] <имя_файла>
```

```
> ./lab1 -s=-1 data.txt
Ошибка: некорректное значение опции -s: '-1'
```

```
> ./lab1 -s=1.5 data.txt
Ошибка: некорректное значение опции -s: '1.5'
```

```
> ./lab1 -s= data.txt
Ошибка: некорректное значение опции -s: ''
```

```
> ./lab1 -s data.txt
Ошибка: некорректное значение опции -s: 'data.txt'
```

Рисунок 3: Результат работы алгоритма при вводе некорректных значений опций

5. ИСХОДНЫЙ КОД АЛГОРИТМА

Программа написана на языке C, для компиляции использовался компилятор gcc.

Исходный код Makefile:

```
1 | CC = gcc
2 | CFLAGS = -O3 -Wall -Wextra -Werror -fanalyzer
3 | TARGET = lab1
4 |
5 | all: $(TARGET)
6 |
7 | $(TARGET): lab1.c
8 |     $(CC) $(CFLAGS) $(TARGET).c -o $(TARGET)
9 |
10 | clean:
11 |     rm -f $(TARGET)
```

Исходный код lab1.c:

```
1 | #include <stdio.h>
2 | #include <fcntl.h>
3 | #include <stdlib.h>
4 | #include <getopt.h>
5 | #include <errno.h>
6 | #include <string.h>
7 |
8 | void printArray(const char *desc, float array[], int size) {
9 |     if (desc) printf("%s", desc); // (2) (неявное !=
NULL + вызов)
10 |     for (int i = 0; i < size; i++) { // (1) + (n+1) + (2n)
= 3n + 2
11 |         printf("%.3f ", array[i]); // (2) (индексация[] +
вызов)
12 |     }
13 |     printf("\n"); // (1)
14 | }
15 | // Итоговая сложность printArray: 2 + 3n + 2 + 2n + 1 = 5n + 5
16 |
17 | int readFromFile(char *filename, float *array, int maxSize) {
18 |     FILE *file = fopen(filename, "r"); // (2) (вызов +
присваивание)
19 |     if (!file) { // (1) (логическое !)
20 |         fprintf(stderr, "Ошибка: не удалось открыть файл %s\n", filename); // (1)
21 |         exit(1); // (1)
22 |     }
23 |
24 |     int n = 0; // (1)
25 |     while (n < maxSize && fscanf(file, "%f", &array[n]) == 1) { // (6) (<, &&, &, [],
вызов, ==) * (n+1)
26 |         n++; // (2) (инкремент) * n
27 |     }
28 |
29 |     fclose(file); // (1)
30 |     return n; // (1)
31 | }
32 | // Итоговая сложность readFromFile (успешное чтение): 2 + 1 + 1 + 6(n+1) + 2n + 1 + 1 =
8n + 12
33 |
34 | int countNums(char *filename) {
35 |     FILE *file = fopen(filename, "r"); // (2) (вызов +
присваивание)
36 |     if (!file) { // (1) (логическое !)
37 |         fprintf(stderr, "Ошибка: не удалось открыть файл %s\n", filename); // (1)
38 |         exit(1); // (1)
39 |     }
40 |     int count = 0; // (1)
41 |     float temp; // (0)
```



```

42 |         while (fscanf(file, "%f", &temp) == 1) { // (3) (&, вызов, ==)
* (n+1)
43 |             count++; // (2) (инкремент) * n
44 |         }
45 |         fclose(file); // (1)
46 |         return count; // (1)
47 |     }
48 |     // Итоговая сложность countNums (успешное чтение): 2 + 1 + 1 + 0 + 3(n+1) + 2n + 1 + 1
= 5n + 9
49 |
50 |     void insertionSort(float array[], int size) {
51 |         for (int i = 1; i < size; i++) { // (1) + (n) +
(2(n-1)) = 3n - 1
52 |             int j = i - 1; // (2) (- и =) * (n-1)
53 |             while (j >= 0 && array[j] > array[j+1]) { // (6) (>=, &&, [],
[, +, >)
54 |                 float tmp = array[j+1]; // (3) (+, [], =)
55 |                 array[j+1] = array[j]; // (4) (+, [], [], =)
56 |                 array[j] = tmp; // (2) ([], =)
57 |                 j--; // (2) (декремент)
58 |             }
59 |             // Сложность одного while в худшем случае: 6(i+1) + (3+4+2+2)*i = 17i + 6
60 |         }
61 |         // Все while: Сумма от i=1 до n-1 (17i + 6) = 17*(n(n-1)/2) + 6(n-1) = 8.5n^2 -
2.5n - 6
62 |     }
63 |     // Итоговая сложность insertionSort: (3n - 1) + 2(n-1) + (8.5n^2 - 2.5n - 6) = 8.5n^2 +
2.5n - 9
64 |
65 |     void bucketSort(float array[], int size) {
66 |         if (size <= 1) return; // (2) (<=, return)
67 |
68 |         float min = array[0], max = array[0]; // (4) ([], =, [], =)
69 |         for (int i = 1; i < size; i++) { // (1) + (n) +
(2(n-1)) = 3n - 1
70 |             if (array[i] < min) min = array[i]; // (4) ([], <, [], =)
71 |             if (array[i] > max) max = array[i]; // (4) ([], >, [], =)
72 |         } // Цикл: (3n - 1) + 8(n-1) = 11n - 9
73 |
74 |         if (min == max) return; // (2) (==, return)
75 |
76 |         int numBuckets = size; // (1)
77 |
78 |         float **buckets = (float**)malloc(numBuckets*sizeof(float*)); // (5) (cast, malloc,
*, sizeof, =)
79 |         if (!buckets) { // (1)
80 |             fprintf(stderr, "Ошибка: не удалось выделить память для корзины\n"); // (1)
81 |             exit(1); // (1)
82 |         }
83 |         for (int i = 0; i < numBuckets; i++) { // (1) + (n+1) + (2n)
= 3n + 2
84 |             buckets[i] = (float*)malloc(size*sizeof(float)); // (6) ([], cast,
malloc, *, sizeof, =)
85 |             if (!buckets[i]) { // (2) ([], !)
86 |                 fprintf(stderr, "Ошибка: не удалось выделить память для корзины %d\n", i);
// (1)
87 |                 exit(1); // (1)
88 |             }
89 |         } // Цикл: (3n + 2) + 8n = 11n + 2
90 |
91 |         int *bucketSizes = (int*)calloc(numBuckets, sizeof(int)); // (4) (cast, calloc,
sizeof, =)
92 |         if (!bucketSizes) { // (1)
93 |             fprintf(stderr, "Ошибка: не удалось выделить память для размеров корзины\n"); //
(1)
94 |             exit(1); // (1)
95 |         }
96 |
97 |         for (int i = 0; i < size; i++) { // (1) + (n+1) + (2n)
= 3n + 2

```

```

98 |         int bucketInd = (int)((array[i] - min)*(numBuckets - 1) / (max - min)); // (8)
99 |         buckets[bucketInd][bucketSizes[bucketInd]] = array[i]; // (5) ([], [], [],
100 |         bucketSizes[bucketInd]++; // (3) ([], ++)
101 |     } // Цикл: 3n + 2 + 8n + 5n + 3n = 19n + 2
102 |
103 |     int ind = 0; // (1)
104 |     for (int i = 0; i < numBuckets; i++) { // (1) + (n+1) + (2n)
105 |         if (bucketSizes[i] > 0) { // (2) ([], >)
106 |             insertionSort(buckets[i], bucketSizes[i]); // (3) ([], [], вызов)
107 |             for (int j = 0; j < bucketSizes[i]; j++) { // (1) + 2(k+1) + (2k)
108 |                 array[ind++] = buckets[i][j]; // (6) (ind++, [], [],
109 |             } // Внутренний цикл: (4k + 3) + 6k = 10k + 3
110 |         }
111 |         free(buckets[i]); // (2) ([], вызов)
112 |     } // Внешний цикл в худшем случае (по 1 элементу в корзине): (3n + 2) + 2n + 3n +
113 |     1n + 4n + 2n + 6n + 2n = 23n + 2 + O(n^2)
114 |     free(buckets); // (1)
115 |     free(bucketSizes); // (1)
116 | }
117 | // Итоговая сложность bucketSort:
118 | // 2 + 4 + (11n - 9) + 2 + 1 + 5 + 1 + (11n + 2) + 4 + 1 + (19n + 2) + 1 + (23n + 2) +
119 | 1 + 1 = 64n + 20 + O(n^2)
120 | int main(int argc, char **argv) {
121 |
122 |     static struct option longopts[] = { // (1)
123 |         {"version", no_argument, 0, 'v'},
124 |         {"help", no_argument, 0, 'h'},
125 |         {"static", required_argument, 0, 's'},
126 |         {"dynamic", no_argument, 0, 'd'},
127 |         {0, 0, 0, 0},
128 |     };
129 |
130 |     int size = 0; // (1)
131 |     int isDynamic = -1; // (1)
132 |
133 |     int opt; // (0)
134 |     while ((opt = getopt_long(argc, argv, "vhs:d", longopts, NULL)) != -1) { // (3)
135 |         if (opt == 'v') { // (1)
136 |             printf("Карманная сортировка (Bucket Sort) v1.0\n"); // (1)
137 |             printf("Информация об авторе:\nПахомов Владимир Юрьевич, гр. N3250, ИСУ
138 |             467036\n"); // (1)
139 |             return 0; // (1)
140 |         } else if (opt == 'h') { // (1)
141 |             printf("Использование: ./lab1 [ОПЦИИ] <имя_файла>\n\n"); // (1)
142 |             printf("Программа для сортировки чисел из файла методом карманной
143 |             сортировки (Bucket Sort)\n\n"); // (1)
144 |             printf("Опции:\n"); // (1)
145 |             printf("    -s, --static=SIZE    Использовать статический массив (VLA)
146 |             заданного размера SIZE\n"); // (1)
147 |             printf("    -d, --dynamic        Использовать динамический массив
148 |             (автоопределение размера файла)\n"); // (1)
149 |             printf("    -h, --help          Показать эту справку и выйти\n"); // (1)
150 |             printf("    -v, --version      Показать информацию о версии и
151 |             выйти\n\n"); // (1)
152 |             printf("Примеры:\n"); // (1)
153 |             printf("./lab1 -s=100 input.txt\n"); // (1)
154 |             printf("./lab1 -d input.txt\n"); // (1)
155 |             return 0; // (1)
156 |         }
157 |         if (opt == 's') { // (1)
158 |             char *endptr; // (0)
159 |             if (optarg[0] == '=') optarg++; // (2) ([], ==) + (2)
160 |             ++ = 4

```

```

155 |         size = strtol(optarg, &endptr, 10); // (3) (&, вызов, =)
156 |         if (optarg == endptr || *endptr != '\0' || size < 0) { // (6) (==, ||, *, !
=, ||, <)
157 |             fprintf(stderr, "Ошибка: некорректное значение опции -s: '%s'\n",
optarg); // (1)
158 |             return 1; // (1)
159 |         }
160 |         isDynamic = 0; // (1)
161 |         // Худшая ветка (s): 1(v) + 1(h) + 1(s) + 4 + 3 + 6 + 1 = 17
162 |     } else if (opt == 'd') { // (1)
163 |         isDynamic = 1; // (1)
164 |     } else {
165 |         fprintf(stderr, "Использование: %s[-s=size | -d] <имя_файла>\n", argv[0]);
// (2)
166 |         return 1; // (1)
167 |     }
168 | } // Цикл while: 3(m+1) + 17m = 20m + 3
169 |
170 | if (optind >= argc) { // (1)
171 |     fprintf(stderr, "Ошибка: не указано имя файла\n"); // (1)
172 |     return 1; // (1)
173 | }
174 |
175 | char *filename = argv[optind]; // (2) ([], =)
176 |
177 | if (isDynamic == 1) { // (1)
178 |     size = countNums(filename); // (2) (вызов, =)
179 |     printf("Используется динамический массив\n"); // (1)
180 |
181 |     float *dynamicArray = malloc(sizeof(float) * size); // (4) (sizeof, *,
вызов, =)
182 |     if (!dynamicArray) { // (1)
183 |         fprintf(stderr, "Ошибка выделения памяти для динамического массива: %s",
strerror(errno)); // (2)
184 |         return 1; // (1)
185 |     }
186 |
187 |     int n = readFromFile(filename, dynamicArray, size); // (2) (вызов, =)
188 |
189 |     printArray("\nИсходный массив из файла:\n", dynamicArray, n); // (1)
190 |     bucketSort(dynamicArray, n); // (1)
191 |     printArray("\nОтсортированный массив:\n", dynamicArray, n); // (1)
192 |
193 |     free(dynamicArray); // (1)
194 |     // Итоговая сложность ветки if: 1 + 2 + 1 + 4 + 1 + 2 + 1 + 1 + 1 + 1 = 15
195 | } else if (isDynamic == 0) { // (1)
196 |     printf("Используется статический массив размера %d\n", size); // (1)
197 |
198 |     float staticArray[size]; // (1)
199 |     int n = readFromFile(filename, staticArray, size); // (2)
200 |
201 |     printArray("\nИсходный массив из файла:\n", staticArray, n); // (1)
202 |     bucketSort(staticArray, n); // (1)
203 |     printArray("\nОтсортированный массив:\n", staticArray, n); // (1)
204 | } else {
205 |     fprintf(stderr, "Выберите режим: --static=SIZE или --dynamic\n"); // (1)
206 |     return 1; // (1)
207 | }
208 |
209 | return 0; // (1)
210 | }
211 | // Итоговая сложность main до вызова функций:
212 | // 3 (инициализация) + (20m + 3) (цикл) + 1 (if) + 2 (filename) + 15 (ветка if) + 1
(return) = 20m + 25
213 |
214 | // ОБЩАЯ СЛОЖНОСТЬ ПРОГРАММЫ (Худший случай - динамический массив):
215 | // T(m, n) = T_main + T_countNums + T_readFromFile + 2 * T_printArray + T_bucketSort
216 | // T(m, n) = (20m + 25) + (5n + 9) + (8n + 12) + 2*(5n + 5) + (64n + 20 + 0(n^2))
217 | // T(m, n) = 20m + 87n + 76 + 0(n^2)
218 | // Асимптотическая сложность: O(m + n^2)

```

ЗАКЛЮЧЕНИЕ

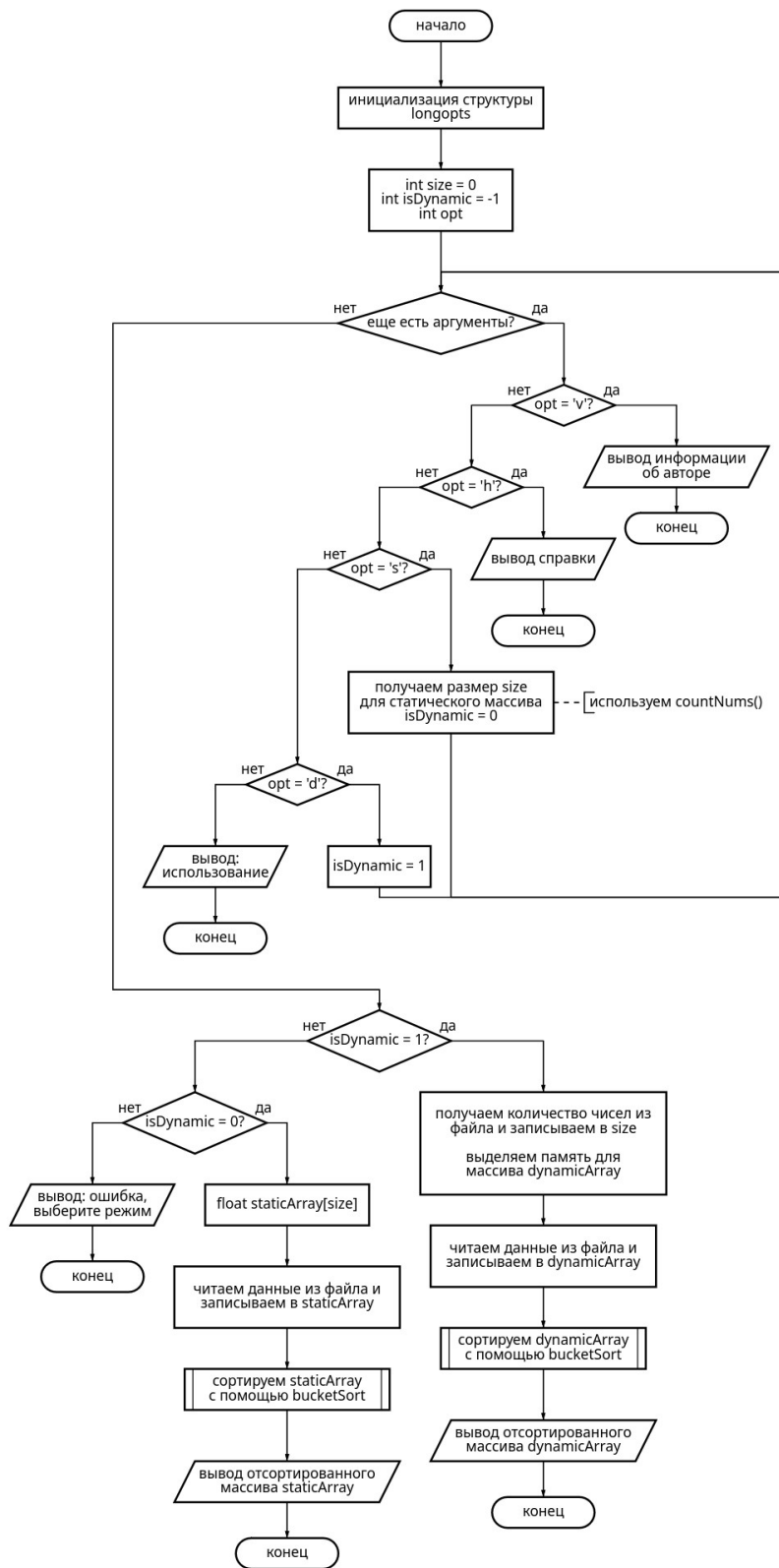
В ходе выполнения лабораторной работы была разработана программа на языке C для сортировки последовательности чисел с помощью алгоритма распределения (карманная сортировка). Для решения задачи также был реализован алгоритм сортировки вставками. Числа считывались из файла и сохранялись в статическом или динамическом массиве по выбору пользователя.

Программа разрабатывалась в среде VS Code, компилировалась с помощью gcc и запускалась в ОС Arch Linux.

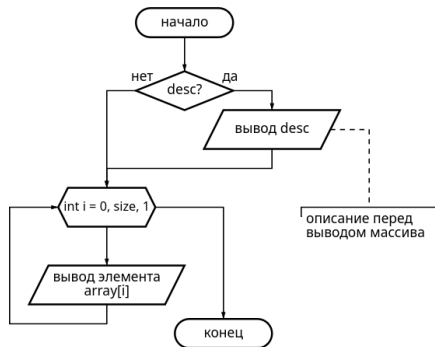
В конце работы была оценена сложность алгоритма путём подсчёта всех элементарных операций, выполняющихся в программе. Тестирование алгоритма показало его корректную работу при различных входных данных.

Выполнение работы позволило закрепить навыки разработки алгоритмов сортировки, а также углубить знания в области структур данных.

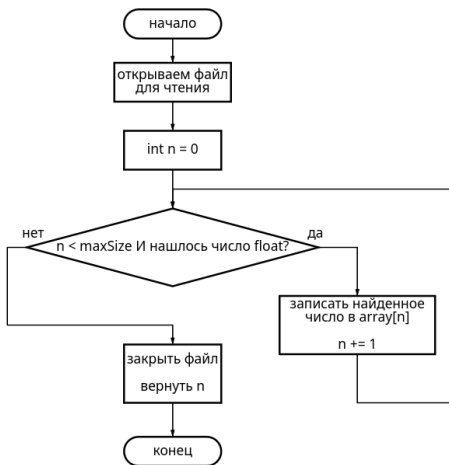
ПРИЛОЖЕНИЕ А



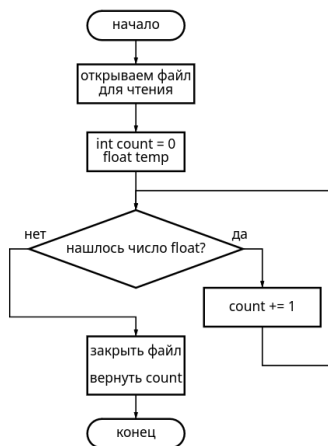
Вывод массива
void printArray(char *desc, float array[], int size)



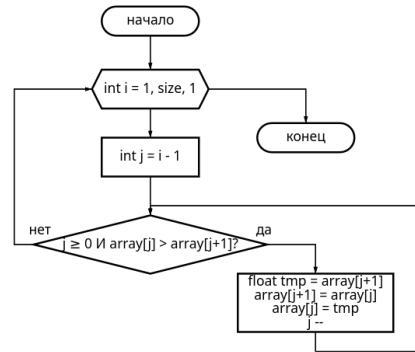
Чтение чисел из файла
int readFromFile(char *filename, float *array, int maxSize)



Подсчет количества чисел в файле
int countNums(char *filename)



сортировка вставками
void insertionSort(float array[], int size)



карманная сортировка
void bucketSort(float array[], int size)

